

SPECIFICATION

goboxd / spec

The technical brief. Everything you need to build a submittable goboxd.

00 What a sandbox does

01 What to ship, by stage

02 The brief

03 API contract

04 Status vocabulary

05 Plug-and-play languages

06 Security

07 Concurrency

08 Repo & tooling

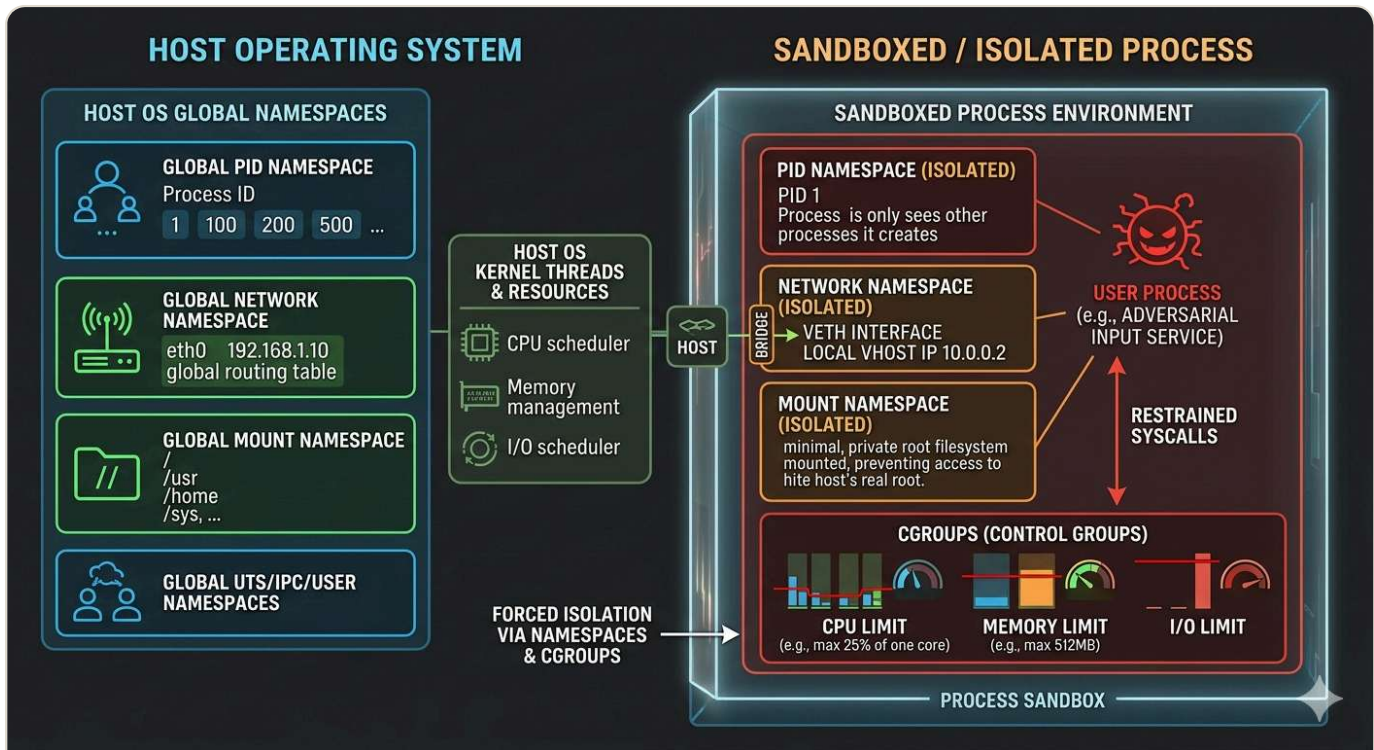
09 Judging

10 Submission

11 Resources

00

What a sandbox actually does



Left: the host operating system. Right: an isolated process inside a sandbox, with its own namespaces for PIDs, networking, and the filesystem, restricted syscalls, and CPU, memory, and I/O capped by cgroups.

A sandbox is a process that the operating system has been tricked into treating as if it were on its own machine. On Linux, the trick is built from two primitives: **namespaces** (which give the process a private view of PIDs, network interfaces, mounts, users, and hostnames) and **cgroups** (which cap how much CPU, memory, and I/O it can use). Add a restricted set of syscalls on top, and a runaway program inside the sandbox can't see, reach, or starve anything outside it.

That's what goboxd is built around. Every `POST /run` request creates a fresh sandbox, runs untrusted code inside it, captures the result, and tears the sandbox down. Stage 1 of the hackathon is about getting that loop working at all. Stage 2 makes it polyglot. Stage 3 makes it survive adversarial pressure under load.

01

What to ship, by stage

Three stages, three escalating bars. Each is judged on what you have at the end of that stage, not on intent.

STAGE 1 · PROTOTYPE

A Go HTTP service that builds and runs in Docker

`/healthz` returns 200

`POST /run` runs two languages end-to-end: one interpreted (Python, Bash, or Node) and one compiled (C, C++, or Java)

Unit tests, a readable README, and a clean commit history

STAGE 2 · POLYGLOT

All seven in-scope languages registered via YAML

`/readyz` and `/info` reflect the registered set

A new language added in under 30 minutes with no Go code change

Per-request limits and flag allow-lists enforced

STAGE 3 · HARDEN & LOAD

5 of 7 security holes closed, with the fixes documented in the PR

Bounded concurrency with a queue, not failures

Benchmarks at 1, 10, 50, and 100 clients in `docs/benchmarks.md` (p50, p95, p99)

Holds up under sustained load on judging day

02

The brief

Build a small HTTP service in Go that runs untrusted code inside an nsjail sandbox and returns per-test results. The service name is `goboxd`, short for "Go sandbox daemon."

A reference Python + Flask implementation is provided so you can read, run, and reverse-engineer the behaviour locally. It is intentionally a bit crusty. Read it as a behaviour spec, not as a guide.

The `goboxd` you build uses plain JSON on the wire, supports modern toolchains only, and is judged primarily on three things:

01

Plug-and-play languages

A clean language registry. We hand you a new language on demo day and you add it in 30 minutes with no Go code change.

02

Concurrency

Bounded, queued, benchmarked. Holds up under sustained load, not just micro-bursts.

03

Security

Close at least 5 of the 7 holes we point you at. Document where you fixed each in the PR.

In scope

C, C++, Java, Python 3, Bash, JavaScript (Node), Verilog. Per-test status. Per-request resource overrides. Per-request build/run flags (validated).

Out of scope

Anything binary on the wire, the reference's evaluation-script mode, the zip language, Python 2, authentication, rate limiting, queues, persistence, web UIs.

You may pick any Go HTTP framework: net/http, chi, echo, gin, whatever. Two-sentence justification in the README.

03

API contract

POST /run

Request body:

```
{
  "language": "cpp",
  "source": "#include <iostream>\nint main(){std::cout<<\"hi\\\";}",
  "source_filename": "solution.cpp",
  "artifact_filename": "solution",
  "build": {
    "limits": { "wall_time_s": 5, "memory_kb": 1048576, "max_processes": 100 },
    "flags": ["-O2"]
  },
  "run": {
    "limits": { "wall_time_s": 3, "memory_kb": 524288, "max_processes": 64 },
    "flags": []
  },
}
```

```

    "tests": [
      { "stdin": "1\n", "expected_stdout": "hi" }
    ]
  }

```

FIELD RULES

- **language** : required. Must match a configured language id, otherwise 400.
- **source** : required, UTF-8. Subject to a server-imposed max size (default 256 KiB).
- **source_filename** , **artifact_filename** : optional. Required only for languages that need them (e.g. Java). Must be a single path component, with no separators, no leading dot, and a length cap.
- **build** and **run** : optional. Each can supply **limits** (a partial override of the language defaults) and **flags** (extra args to the build or run command). Missing fields fall back to language defaults
- **flags** : an array of strings. The server filters these against a per-language allow-list and rejects unsafe flags with a 400. See [Security](#).
- **tests** : required, at least one entry. Each **stdin** is fed to the program, and **expected_stdout** is compared to its stdout.

Response (200 once the run completes, regardless of user-code outcome):

```

{
  "status": "wrong_output",
  "build": {
    "status": "ok",
    "stdout": "",
    "stderr": "",
    "duration_ms": 412
  },
  "tests": [
    {
      "status": "wrong_output",
      "stdout": "HI",
      "stderr": "",

```

COPY

```

        "duration_ms": 38,
        "memory_peak_kb": 8192
    }
]
}

```

ERRORS

Errors that prevent any execution (bad JSON, unknown language, oversize body, malformed filename, disallowed flag) return HTTP 400 :

```

{
  "error": {
    "code": "invalid_filename",
    "message": "source_filename must be a single path component"
  }
}

```

[COPY](#)

5xx is reserved for server failures (sandbox setup error, nsjail not found, disk full). Never return 5xx because the user code crashed.

GET /healthz

Liveness. Cheap, no dependencies. Returns 200 {"status":"ok"} if the process is up.

GET /readyz

Readiness. Returns 200 only if the nsjail binary is executable and every configured language's compiler/runtime resolves and passes its smoke probe (--version or a per-language override). On failure, returns 503 with a per-language breakdown:

```

{
  "status": "degraded",

```

[COPY](#)


```

"nsjail": { "ok": true, "version": "3.4" },
"languages": {
  "py3": { "ok": true, "version": "Python 3.11.2" },
  "java": { "ok": false, "error": "javac not found at /usr/bin/javac" }
}
}

```

GET /info

Always 200 .

COPY

```

{
  "build_info": { "version": "0.1.0", "commit": "abc1234", "go_version": "go1.22."
  "nsjail": { "path": "/usr/bin/nsjail", "version": "3.4" },
  "languages": [
    {
      "id": "py3",
      "name": "Python 3",
      "version": "Python 3.11.2",
      "default_run_limits": { "wall_time_s": 9, "memory_kb": 102400, "max_process
    }
  ],
  "limits": {
    "max_source_bytes": 262144,
    "max_tests": 50,
    "max_concurrent_jobs": 16
  },
  "stats": {
    "in_flight_jobs": 3,
    "jobs_total": 41892,
    "jobs_failed_internal": 4,
    "last_internal_error_at": "2026-05-04T11:22:09Z",
    "disk_free_bytes_jail_dir": 53687091200
  }
}

```

Cgroup memory pressure or Go GC stats are welcome but not required.

04

Status vocabulary

SCOPE	ALLOWED VALUES
<code>build.status</code>	<code>ok</code> · <code>failed</code> · <code>internal_error</code>
<code>test.status</code>	<code>accepted</code> · <code>wrong_output</code> · <code>output_whitespace_mismatch</code> · <code>time_exceeded</code> · <code>memory_exceeded</code> · <code>runtime_error</code> · <code>not_executed</code> · <code>internal_error</code>
Top-level	<code>accepted</code> · <code>build_failed</code> · <code>wrong_output</code> · <code>output_whitespace_mismatch</code> · <code>time_exceeded</code> · <code>memory_exceeded</code> · <code>runtime_error</code> · <code>internal_error</code>

Top-level rule: top-level status is `accepted` only if `build.status == ok` and every test is `accepted` . Otherwise it is the first non-accepted status in test order. If the build fails, top-level is `build_failed` and every `tests[].status` is `not_executed` .

05

Plug-and-play languages

Adding a language should be one YAML block and one PR. No Go code change unless the language needs custom logic.

Recommended shape. You may refine it; just document whatever you change.

COPY

```
languages:
- id: py3
  name: Python 3
  source_filename: solution.py
  run:
    cmd: /usr/bin/python3
    args: ["{{source}}"]
    limits: { wall_time_s: 9, memory_kb: 102400, max_processes: 100 }

- id: cpp
  name: C++
  source_filename: solution.cpp
  artifact: solution
  build:
    cmd: /usr/bin/g++
    args: ["{{flags}}", "-o", "{{artifact}}", "{{source}}"]
    limits: { wall_time_s: 3, memory_kb: 1048576, max_processes: 100 }
    flag_allowlist: ["-O0", "-O1", "-O2", "-O3", "-Wall", "-Wextra", "-std=*"]
  run:
    cmd: ./{{artifact}}
    limits: { wall_time_s: 3, memory_kb: 524288, max_processes: 64 }

- id: java
  name: Java
  source_filename_strategy: from_request
  artifact_filename_strategy: from_request
  build:
    cmd: /usr/bin/javac
    args: ["{{flags}}", "{{source}}"]
    limits: { wall_time_s: 6, memory_kb: 102400, max_processes: 100 }
  run:
    cmd: /usr/bin/java
    args: ["{{artifact}}"]
    limits: { wall_time_s: 6, memory_kb: 102400, max_processes: 100 }
```

What we look at on demo day

We'll hand you a language we don't already support (Rust, Go, Kotlin, Ruby, etc.) and ask you to register it. You should be able to do it in 30 minutes or less, with no code change. Just a YAML edit, an installation script during the Docker build, and a smoke test. We'll watch you do it.

We also look at: clear placeholder semantics, startup-time validation that fails loudly with a useful error, and `/readyz` and `/info` reflecting the registered set automatically.

06

Security

The reference has the holes below. Fix at least five in your Go version. Each extra one above five counts as a bonus. In your PR description, list which ones you closed and link to the `file:line` where you closed each.

1. Path traversal via filename.

The reference joins the jail directory with a client-supplied filename and writes the file with no validation. Values like `../../etc/passwd` or absolute paths escape the jail dir on the host. Validate strictly.

2. Shell-style directory commands.

The reference creates and deletes per-request directories by string-formatting commands and running them through a shell. Use the language's filesystem APIs; audit every path-handling line.

3. Compiler-flag injection.

The reference appends arbitrary client-supplied args to `gcc`, `g++`, `javac`. Flags like `-fplugin=...`, `-x c`, `-B...`, `--specs=...`, `-Wl,...`, `@response_file` give compile-time code execution. Per-language allow-list, reject the rest with `400`.

4. No request size limits.

Source size, test count, per-test stdin and expected size, captured stdout and stderr: all unbounded. Cap them at the HTTP layer and at the sandbox layer (`rlimit_fsize` , capped reads).

5. UID collisions under load.

The reference picks a UID from a 30k-wide range and retries three times on collision. Use a process-unique scheme (atomic counter + PID + random suffix, or a `tempdir` API) and never reuse a directory.

6. Unbounded child output.

Reading the full child stdout into memory lets a runaway program OOM the host. Cap captured output and truncate with a marker.

7. Stale jail directories.

A panic between create and cleanup leaks the directory. Cleanup must run on every exit path (Go: `defer` at the right scope) plus a startup sweep of orphans older than N minutes.

07

Concurrency

The judging weight on concurrency is high. We want evidence, not vibes.

- A bounded global concurrency limit, configurable (env var or YAML), default `runtime.NumCPU()` . When the limit is reached, requests queue rather than fail.
- A small load-test script in the repo (`hey` , `vegeta` , `k6` , or your own).
- A `docs/benchmarks.md` showing requests/sec and p50/p95/p99 for the trivial "Hello World, py3" case at 1, 10, 50, and 100 concurrent clients on whatever box you measured on. Numbers from a clean Docker run, not from a debugger.

- Per-request CPU and wall-clock time captured and visible somewhere. Logs are fine.

Don't optimise micro-benchmarks while starving under sustained load. We will do a sustained-load run during judging.

08

Repo structure and tooling

The repo at submission time should be self-contained and run from a fresh clone in under 10 minutes.

- **Dockerfile.** Builds the binary, builds nsjail from source at image-build time, and installs every supported language toolchain. The container is the unit of "it works." nsjail must be added as a git submodule (preferred) or fetched in the Dockerfile, pinned to upstream tag `3.4`, and built inside the image. Do not bundle a prebuilt binary. Do not assume the host has nsjail installed. Do not install it from a package archive. Expect to see

```
git submodule add https://github.com/google/nsjail external/nsjail
```

 (or equivalent) and a build step in the Dockerfile.
- **Docker Compose.** Brings up the server for local development with a single command.
- **Makefile.** Every common operation has a target. At minimum: `make build`, `make run`, `make test`, `make integration`, `make load`, `make lint`. No bare `go run` instructions in the README.
- **The docs/ folder.** Anything longer than a paragraph lives here as its own file: `docs/api.md`, `docs/languages.md`, `docs/security.md`, `docs/benchmarks.md`, `docs/architecture.md`.
- **README.md.** Short. What it is, how to run, where the docs are. Write it as a human wrote it. No AI filler. No "elegant", "robust", "seamlessly", or "leverage". No emoji decoration. No marketing copy. If a sentence wouldn't survive a code review, cut it.

- **Tests.** Unit tests on everything that doesn't need nsjail: config loading, filename validation, flag allow-listing, status mapping, output truncation. At least one end-to-end test per in-scope language.
- **Format and linting.** `go vet` and `staticcheck` (or `golangci-lint`) must be clean.

Everything runs in Docker. If we can't `docker build` and `docker run` your repo and have `/healthz` return `200`, you're not finished.

09

Judging

On judging day, we run a private corpus of `POST /run` requests against your container, plus the demo-day language addition, plus a sustained-load run.

CRITERION	WEIGHT
Plug-and-play language model (YAML, ease of adding, demo-day add)	25%
Concurrency: architecture, benchmarks, sustained-load behavior	20%
API contract conformance (every field, every status, every error case)	15%
Security holes closed (5 required, more = bonus)	15%
Code quality, readability, tests	10%
Languages beyond the in-scope seven	10%
Health endpoints (<code>/healthz</code> , <code>/readyz</code> , <code>/info</code>): accuracy and completeness	5%

Bonuses

- **Languages beyond the seven in scope.** Rust, Go, Kotlin, C#, Ruby, Lua, OCaml, Swift, Zig. Each one that passes its smoke probe on `/readyz` is worth one point.
 - **Closing more than five of the seven security holes.** Each extra closure counts for half the score of a required one.
 - **Structured request logs.** One JSON line per request, with request id, language, durations, and status.
 - **A clean architecture doc at** `docs/architecture.md` . Good enough that we could hand it to a new engineer on their first day.
-

10

Submission

Submission repo: `github.com/thesouldev/goboxd` . Base branch: `master` . The workflow is plain open-source: fork, branch, PR. We do not add you as a collaborator.

1. **Register through the Paradox listing.** That is how we know you exist.
2. **Fork the submission repo.** Fork `thesouldev/goboxd` to your own GitHub account, or to a team account if you are working as a pair.
3. **Create a team branch.** In your fork, branch off `master` as `team/<your-team-name>` .
4. **Work on that branch.** Commit and push to your fork as often as you like. Keep the history clean. Avoid one giant commit for the whole feature; we want to see how you worked.
5. **Open a PR before the deadline.** Open it from `<your-fork>:team/<your-team-name>` back to `thesouldev/goboxd:master` . Mark it ready-for-review.
6. **PR description.** Include: team members, framework choice in one sentence with the reason, how to run locally in one paragraph pointing at the Makefile, the security

holes you closed with `file:line` links, languages supported, and a link to `docs/benchmarks.md` .

Late PRs and post-deadline force-pushes are not considered.

11

Resources

Submission repo. github.com/thesouldev/goboxd

Reference Python + Flask implementation (read-only). [Drive link](#)

Sample request and reply pairs. Inside the reference repo, under `tests/`

Language install scripts used by the reference. Inside the reference repo, under `scripts/lang_install/`

Google nsjail (the upstream sandbox we wrap). github.com/google/nsjail. Pin tag `3.4` .

Questions. GitHub Discussions

goboxd · A SEEK hackathon at Paradox, IIT Madras · 2026

[Overview](#) [Spec](#) [Discussions](#) [Submission repo](#)